

Projet Intégrateur : PacMan en réseau

Thomas Coutant | Benoît Litampha | Auriane Peter

Université Marie & Louis Pasteur, UFR Sciences et Techniques

Licence CMI Informatique – 3^{ème} année
2025–2026

Encadrant : Julien Bernard



Plan

- 1 Présentation et organisation
- 2 Serveur
- 3 Client
- 4 Réseau
- 5 Démonstration

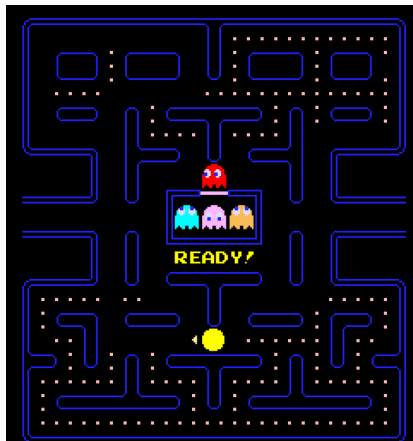
Introduction

Jeu d'arcade apparu dans les **années 1980**

- Pac-Man (le joueur) évolue dans un **labyrinthe**
- Objectif : manger toutes les **pac-gommes**
- Éviter les **fantômes**
- Certaines pac-gommes permettent de **manger les fantômes**

Objectifs du projet :

- Jeu fonctionnel en réseau
- Génération procédurale du plateau
- Intelligence artificielle des fantômes



Jeu original Pac-Man

Adaptation du projet :

- 1 joueur Pac-Man
- Plusieurs joueurs Fantômes
- Fantômes contrôlés par **IA** si joueurs insuffisants
- Cabane comme lieu d'apparition des fantômes

Déroulement d'une partie :

Connexion > Liste des Salons > Salon > Jeu > Fin de jeu

Plan

1 Présentation et organisation

2 Serveur

3 Client

4 Réseau

5 Démonstration

- **Git**
- **C++** avec le framework **GameDev**
- **Réunions régulières** : toutes les 1 à 2 semaines
- **Communication quotidienne** avec **Discord**



■ Auriane Peter

- Développement du client
- Interface et interactions joueur

■ Benoît Litampha

- Communication client–serveur
- Support sur client et serveur

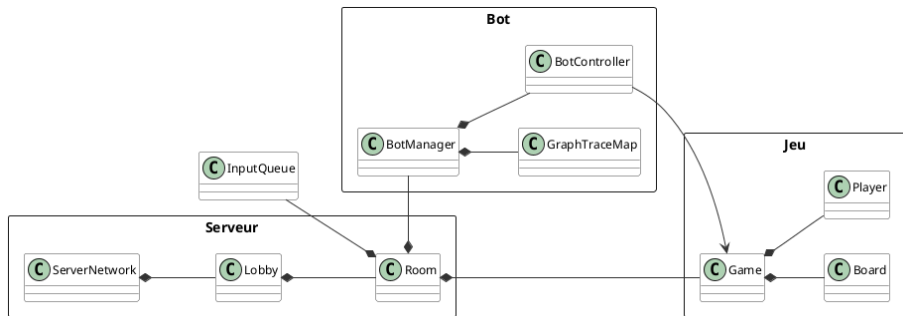
■ Thomas Coutant

- Développement du serveur
- Conception de la logique du jeu

Plan

- 1 Présentation et organisation
- 2 **Serveur**
- 3 Client
- 4 Réseau
- 5 Démonstration

Architecture du serveur



Client → Serveur → Hall → Salon → Game

Architecture modulaire

Architecture

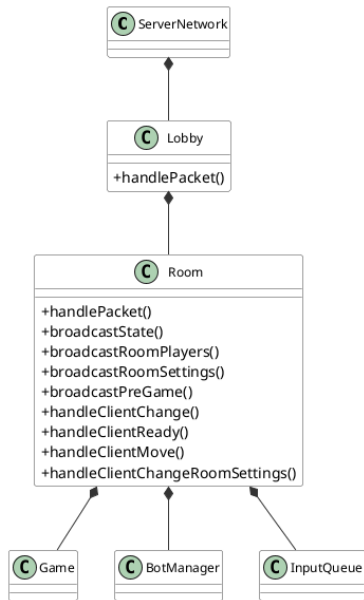
- Organisation **modulaire**
- Séparation des responsabilités
- Lisibilité et maintenabilité

Couches

- Réseau : Serveur
- Organisation : Hall
- Partie : Salon
- Logique : Jeu

Rôle central

- Coordination des composants
- Gestion des joueurs
- Synchronisation des états



Boucle de jeu

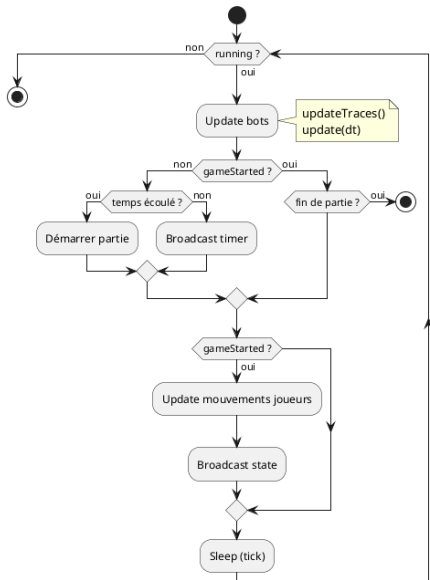
- Serveur **autoritaire**
- Affichage **côté client**
- Anti-triche et synchronisation

Tick serveur

- Mise à jour **périodique**
- Thread serveur :
 - Positions
 - Collisions
 - Scores et timers

États

- Pré-jeu
- Partie en cours
- Fin de partie



IA des fantômes - Principe

Objectif

- Adaptation au nombre de joueurs
- Intelligence collective

Inspiration : fourmis

- Phéromones
- Communication indirecte
- Suivi de traces

Implémentation

- Traces sur le plateau
- Paramètres :
 - Intensité (décroissance)
 - Propriétaire
 - Couleur

```
struct Trace {  
    TraceType type;  
    uint32_t ownerId;  
    float intensity;  
};
```

Rouge : Pac-Man

| Bleu : fantômes

IA des fantômes - Prise de décision

Score

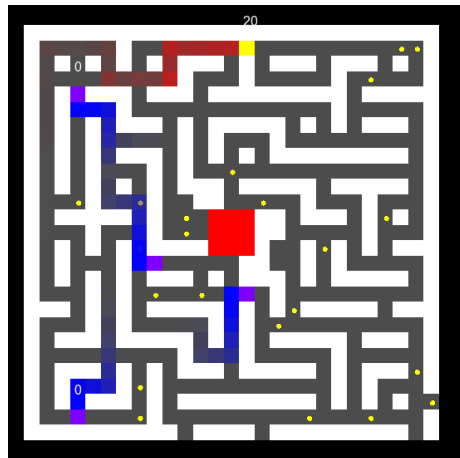
$$\text{Score} = \text{Attraction} - \text{Répulsions}$$

Influences

- Pac-Man -> Attraction
- Fantômes -> Répulsion
- Répulsion ++ pour lui-même
- Vision -> priorité Pac-Man
- Tracking -> BFS

Intégration serveur

- Bots = joueurs
- Envoi d'inputs
- InputQueue -> Salon



Génération procédurale du plateau

Algorithme

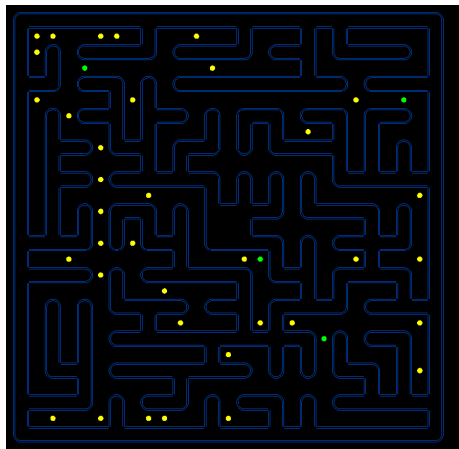
- Variante de Prim

Étapes

- Plateau = murs
- Case de départ
- Liste des murs adjacents
- Ouverture aléatoire

Propriétés

- Connexe
- Aucune zone isolée

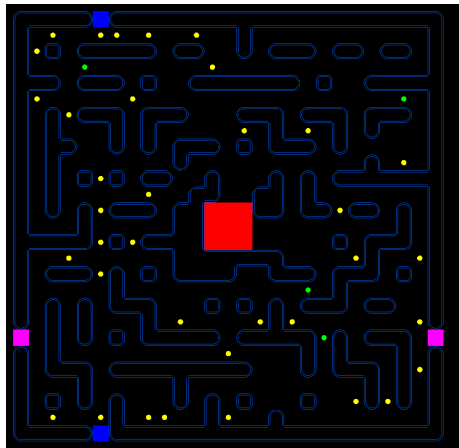


Contraintes

- Zone centrale : hut 3×3
- Accès $\rightarrow$ coins de départ

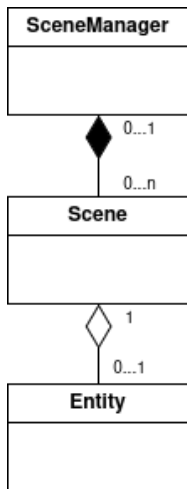
Améliorations

- Cycles
- Moins de culs-de-sac
- Portails latéraux



Plan

- 1 Présentation et organisation
- 2 Serveur
- 3 Client**
- 4 Réseau
- 5 Démonstration



Fonctionnement du SceneManager

Le SceneManager gère quelle scène s'affiche puis :

- Chaque étape de jeu : **une scène**
- La scène se **gère elle même**
- **Séparation** entre logique / rendu

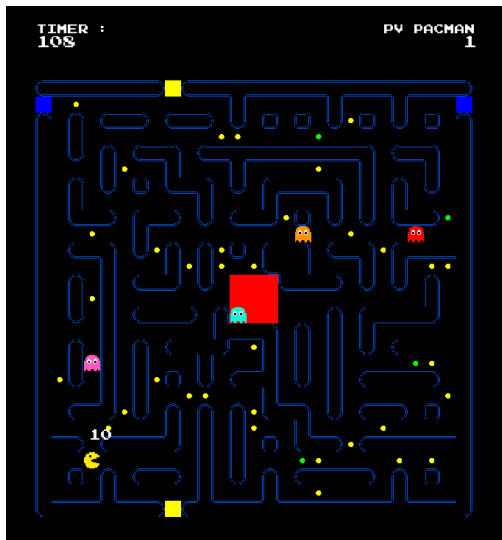
Met aussi en place le réseau

Système d'Événements :

- Mapping clavier -> actions (up, down, left, right)
- Passage de la souris
- Clics de la souris
- Redimensionnement de la fenêtre

Une partie est **envoyé au serveur**





Les scènes envoient les messages au serveur

Deux modèles :

- événementiel (menus, hall)
- périodique (mouvements en jeu)

Principe d'envoi est **le même**

L'envoi transmet l'intention du client au serveur

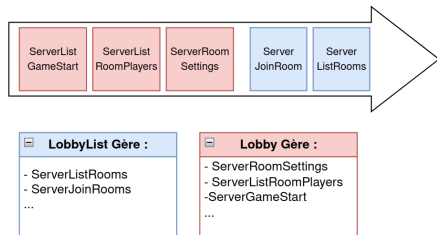
Reception

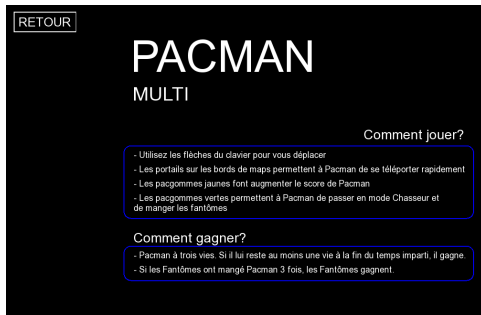
Réception dans un **Thread dédié**

Paquets stockés dans une file partagée :

- Scène consomme paquets **selon leur type**
- Gère seulement ce **qui les concerne**

La scène se met à jour en fonction de ce qu'elle a reçu





Chaque scène a une entité

Les entités ont **peu de travail** :

- Traduit la logique en **éléments graphiques concrets**
- Interagit directement avec le client : **clics client** -> **scène**

Plan

- 1 Présentation et organisation
- 2 Serveur
- 3 Client
- 4 Réseau**
- 5 Démonstration

Communication avec socket

- Choix entre UDP et TCP : TCP choisit pour sa fiabilité
- `gf::TcpListener` et `gf::TcpSocket` pour la connexion/communication

Classes Spécialisées

- **ServerNetwork** pour la gestion des nouveaux clients et des packets (envoi/réception)
- **ClientNetwork** pour la gestion des packets (réception)

Multi-Threading

- Fonctions de **ServerNetwork** et **ClientNetwork** exécuté dans un thread
- Dépilement des packets dans les boucles principales

Structures communes

- Utilisées à la fois par le client et le serveur
- Peuvent être générées dynamiquement par le serveur

Structures de Communication

- Utilisées pour les échanges réseaux
- Un champ `gf::id` pour les identifier lors de la désérialisation
- Nomenclature spécifique :
 - *ServerXXX* = envoyée par le serveur
 - *ClientXXX* = envoyée par le client

```
struct PlayerData
{
    uint32_t id;
    float x, y;
    uint32_t color;
    std::string name;
    PlayerRole role;
    int score;
    bool ready = false;
    unsigned int hp = 0;
};
```

```
struct ServerListRoomPlayers
{
    static constexpr gf::Id type =
        "ServerListRoomPlayers"
        _id;
    std::vector<PlayerData>
        players;
};
```


Opérateur

- Opérateur | avec un type *Archive*

Sérialisation/Envoi

- Fonction `gf::Packet::is(obj)` pour sérialiser en suite d'octet
- Fonction `gf::TcpSocket::sendPacket(packet)` pour envoyer le packet

Désérialisation

- Switch case pour identifier le type
- Fonction `gf::Packet::<typename>as()` pour désérialiser

```
template <typename Archive>
Archive &operator|(Archive &ar,
                  PlayerData &data)
{
    return ar | data.id
              | data.x
              | data.y
              | data.color
              | data.name
              | data.role
              | data.score
              | data.ready
              | data.hp;
}
```

Exemple concret : Envoi de la liste des joueurs

Dans le serveur

```
void Salon::broadcastRoomPlayers() {  
    ServerListRoomPlayers list;  
    for (uint32_t pid : players) {  
        PlayerData pdata;  
        ...  
        list.players.push_back(pdata);  
    }  
    gf::Packet packet;  
    packet.is(list);  
    for (uint32_t pid : players) {  
        network.send(pid, packet);  
    }  
}
```

Dans le client

```
void LobbyScene::doUpdate(gf::Time)  
{  
    gf::Packet packet;  
    while (m_game.tryPopPacket(packet)) {  
        switch (packet.getType()) {  
            case ServerRoomSettings::type:  
                ...  
            case ServerListRoomPlayers::type:  
                {  
                    auto data =  
                        packet.as<ServerListRoomPlayers>();  
                    setPlayers(data.players);  
                    break;  
                }  
            ...  
        }  
    }  
}
```

Objectif principal atteint

- Communication client/serveur
- Intelligence artificielle des fantômes
- Génération procédural de la carte

Aquisition et renforcement des compétences

- Programmation réseau, multi-threadé
- Modélisation des systèmes
- Travail en équipe

Perspectives

- Plusieurs rôles fantômes
- Intelligence artificielle pour Pac-Man
- Chambre privée

**Merci de votre attention.
Des question ?**